JavaScript is disabled on your browser.

Java SE 27 & JDK 27

Class PushbackInputStream

java.lang.Object
    java.io.InputStream
        java.io.FilterInputStream
            java.io.PushbackInputStream

All Implemented Interfaces:

Closeable, AutoCloseable

```
public class PushbackInputStream
extends FilterInputStream
```

A `PushbackInputStream` adds functionality to another input stream, namely the ability to "push back" or "unread" bytes, by storing pushed-back bytes in an internal buffer. This is useful in situations where it is convenient for a fragment of code to read an indefinite number of data bytes that are delimited by a particular byte value; after reading the terminating byte, the code fragment can "unread" it, so that the next read operation on the input stream will reread the byte that was pushed back. For example, bytes representing the characters constituting an identifier might be terminated by a byte representing an operator character; a method whose job is to read just an identifier can read until it sees the operator and then push the operator back to be re-read.

Since:

1.0

Field Summary

Fields

Modifier and Type	Field	Description
`protected byte[]`	**buf**	The pushback buffer.
`protected int`	**pos**	The position within the pushback buffer from which the next byte will be read.

Fields declared in class FilterInputStream

in

Constructor Summary

Constructors

Constructor	Description
`PushbackInputStream (InputStream in)`	Creates a `PushbackInputStream` with a 1-byte pushback buffer, and saves its argument, the input stream `in`, for later use.
`PushbackInputStream (InputStream in, int size)`	Creates a `PushbackInputStream` with a pushback buffer of the specified `size`, and saves its argument, the input stream `in`, for later use.

Method Summary

All Methods | Instance Methods

Modifier and Type	Method	Description
`int`	`available()`	Returns an estimate of the number of bytes that can be read (or skipped over) from this input stream without blocking by the next invocation of a method for this input stream.
`void`	`close()`	Closes this input stream and releases any system resources associated with the stream.
`void`	`mark(int readlimit)`	Marks the current position in this input stream.
`boolean`	`markSupported()`	Tests if this input stream supports the `mark` and `reset` methods, which it does not.
`int`	`read()`	Reads the next byte of data from this input stream.
`int`		

	read(byte[] b, int off, int len)	Reads up to len bytes of data from this input stream into an array of bytes.
void	**reset**()	Repositions this stream to the position at the time the mark method was last called on this input stream.
long	**skip**(long n)	Skips over and discards n bytes of data from this input stream.
void	**unread**(byte[] b)	Pushes back an array of bytes by copying it to the front of the pushback buffer.
void	**unread**(byte[] b, int off, int len)	Pushes back a portion of an array of bytes by copying it to the front of the pushback buffer.
void	**unread**(int b)	Pushes back a byte by copying it to the front of the pushback buffer.

Methods declared in class FilterInputStream

read

Methods declared in class InputStream

nullInputStream, readAllBytes, readNBytes, readNBytes, skipNBytes, transferTo

Methods declared in class Object

`clone`, `equals`, `finalize`, `getClass`, `hashCode`, `notify`, `notifyAll`, `toString`, `wait`, `wait`, `wait`

Field Details

buf

```
protected byte[] buf
```

The pushback buffer.

Since:

1.1

pos

```
protected int pos
```

The position within the pushback buffer from which the next byte will be read. When the buffer is empty, `pos` is equal to `buf.length`; when the buffer is full, `pos` is equal to zero.

Since:

1.1

Constructor Details

PushbackInputStream

```
public PushbackInputStream(InputStream in,
                           int size)
```

Creates a `PushbackInputStream` with a pushback buffer of the specified `size`, and saves its argument, the input stream `in`, for later use. Initially, the pushback buffer is empty.

Parameters:

`in` - the input stream from which bytes will be read.

`size` - the size of the pushback buffer.

Throws:

`IllegalArgumentException` - if `size <= 0`

Since:

1.1

PushbackInputStream

```
public PushbackInputStream(InputStream in)
```

Creates a `PushbackInputStream` with a 1-byte pushback buffer, and saves its argument, the input stream `in`, for later use. Initially, the pushback buffer is empty.

Parameters:

`in` - the input stream from which bytes will be read.

Method Details

read

```
public int read()
         throws IOException
```

Reads the next byte of data from this input stream. The value byte is returned as an `int` in the range `0` to `255`. If no byte is available because the end of the stream has been reached, the value `-1` is returned. This method blocks until input data is available, the end of the stream is detected, or an exception is thrown.

This method returns the most recently pushed-back byte, if there is one, and otherwise calls the `read` method of its underlying input stream and returns whatever value that method returns.

Overrides:

`read` in class `FilterInputStream`

Returns:

the next byte of data, or `-1` if the end of the stream has been reached.

Throws:

`IOException` - if this input stream has been closed by invoking its `close()` method, or an I/O error occurs.

See Also:

`InputStream.read()`

read

```
public int read(byte[] b,
                int off,
                int len)
         throws IOException
```

Reads up to `len` bytes of data from this input stream into an array of bytes. This method first reads any pushed-back bytes; after that, if fewer than `len` bytes have been read then it reads from the underlying input stream. If `len` is not zero, the method blocks until at least 1 byte of input is available; otherwise, no bytes are read and `0` is returned.

Overrides:

`read` in class `FilterInputStream`

Parameters:

`b` - the buffer into which the data is read.

`off` - the start offset in the destination array `b`

`len` - the maximum number of bytes read.

Returns:

the total number of bytes read into the buffer, or `-1` if there is no more data because the end of the stream has been reached.

Throws:

`NullPointerException` - If `b` is `null`.

`IndexOutOfBoundsException` - If `off` is negative, `len` is negative, or `len` is greater than `b.length - off`

`IOException` - if this input stream has been closed by invoking its `close()` method, or an I/O error occurs.

See Also:

`InputStream.read(byte[], int, int)`

unread

```
public void unread(int b)
            throws IOException
```

Pushes back a byte by copying it to the front of the pushback buffer. After this method returns, the next byte to be read will have the value `(byte)b`.

Parameters:

`b` - the `int` value whose low-order byte is to be pushed back.

Throws:

`IOException` - If there is not enough room in the pushback buffer for the byte, or this input stream has been closed by invoking its `close()` method.

unread

```
public void unread(byte[] b,
                   int off,
                   int len)
            throws IOException
```

Pushes back a portion of an array of bytes by copying it to the front of the pushback buffer. After this method returns, the next byte to be read will have the value `b[off]`, the byte after that will have the value `b[off+1]`, and so forth.

Parameters:

`b` - the byte array to push back.

`off` - the start offset of the data.

`len` - the number of bytes to push back.

Throws:

`NullPointerException` - If `b` is `null`.

`IOException` - If there is not enough room in the pushback buffer for the specified number of bytes, or this input stream has been closed by invoking its `close()` method.

Since:

1.1

unread

```
public void unread(byte[] b)
            throws IOException
```

Pushes back an array of bytes by copying it to the front of the pushback buffer. After this method returns, the next byte to be read will have the value `b[0]`, the byte after that will have the value `b[1]`, and so forth.

Parameters:

`b` - the byte array to push back

Throws:

`NullPointerException` - If `b` is `null`.

`IOException` - If there is not enough room in the pushback buffer for the specified number of bytes, or this input stream has been closed by invoking its `close()` method.

Since:

1.1

available

```
public int available()
             throws IOException
```

Returns an estimate of the number of bytes that can be read (or skipped over) from this input stream without blocking by the next invocation of a method for this input stream. The next invocation might be the same thread or another thread. A single read or skip of this many bytes will not block, but may read or skip fewer bytes.

The method returns the sum of the number of bytes that have been pushed back and the value returned by `available`.

Overrides:

`available` in class `FilterInputStream`

Returns:

the number of bytes that can be read (or skipped over) from the input stream without blocking.

Throws:

`IOException` - if this input stream has been closed by invoking its `close()` method, or an I/O error occurs.

See Also:

`FilterInputStream.in`, `InputStream.available()`

skip

```
public long skip(long n)
          throws IOException
```

Skips over and discards `n` bytes of data from this input stream. The `skip` method may, for a variety of reasons, end up skipping over some smaller number of bytes, possibly zero. If `n` is negative, no bytes are skipped.

The `skip` method of `PushbackInputStream` first skips over the bytes in the pushback buffer, if any. It then calls the `skip` method of the underlying input stream if more bytes need to be skipped. The actual number of bytes skipped is returned.

Overrides:

`skip` in class `FilterInputStream`

Parameters:

`n` - the number of bytes to be skipped.

Returns:

the actual number of bytes skipped.

Throws:

`IOException` - if the stream has been closed by invoking its `close()` method, `in.skip(n)` throws an IOException, or an I/O error occurs.

Since:

1.2

See Also:

`FilterInputStream.in`, `InputStream.skip(long n)`

markSupported

```
public boolean markSupported()
```

Tests if this input stream supports the `mark` and `reset` methods, which it does not.

Overrides:

`markSupported` in class `FilterInputStream`

Returns:

`false`, since this class does not support the `mark` and `reset` methods.

See Also:

`InputStream.mark(int)`, `InputStream.reset()`

mark

```
public void mark(int readlimit)
```

Marks the current position in this input stream.

The `mark` method of `PushbackInputStream` does nothing.

Overrides:

`mark` in class `FilterInputStream`

Parameters:

`readlimit` - the maximum limit of bytes that can be read before the mark position becomes invalid.

See Also:

`InputStream.reset()`

reset

```
public void reset()
           throws IOException
```

Repositions this stream to the position at the time the `mark` method was last called on this input stream.

The method `reset` for class `PushbackInputStream` does nothing except throw an `IOException`.

Overrides:

`reset` in class `FilterInputStream`

Throws:

`IOException` - if this method is invoked.

See Also:

`InputStream.mark(int)`, `IOException`

close

```
public void close()
           throws IOException
```

Closes this input stream and releases any system resources associated with the stream. Once the stream has been closed, further read(), unread(), available(), reset(), or skip() invocations will throw an IOException. Closing a previously closed stream has no effect.

Specified by:

`close` in interface `AutoCloseable`

Specified by:

`close` in interface `Closeable`

Overrides:

`close` in class `FilterInputStream`

Throws:

`IOException` - if an I/O error occurs.

See Also:

`FilterInputStream.in`

Report a bug or suggest an enhancement
For further API reference and developer documentation see the Java SE Documentation, which contains more detailed, developer-targeted descriptions with conceptual overviews, definitions of terms, workarounds, and working code examples. Other versions.
Java is a trademark or registered trademark of Oracle and/or its affiliates in the US and other countries.
Copyright © 1993, 2026, Oracle and/or its affiliates, 500 Oracle Parkway, Redwood Shores, CA 94065 USA.
All rights reserved. Use is subject to license terms and the documentation redistribution policy.

Scripting on this page tracks web page traffic, but does not change the content in any way.